AI Engineering and Claude Architect · Part IV, Advanced

Advanced AI Topics, Lesson 2

Self-Hosting Open Models and Quantization

"The model is free, so hosting it must be cheaper." At today's small-model API prices, the fixed cost of two servers and one engineer buys about 155 million requests a month. Cost is rarely the honest reason to self-host.

Module	AI11 of 13
Lesson	2 of 5
Prerequisites	AI02 L3 and L5, HLD M21D, AI10 L2, AI11 L1
Exam relevance	Out of scope
Reading time	~30 minutes

Contents

1. Self-Hosting and Quantization

Lesson 2 of 5 · Advanced AI Topics

Self-Hosting and Quantization

Key idea

Self-host for control, and be suspicious of cost as the reason, because idle GPUs bill by the hour while an API bills by the token. Size memory for weights and KV cache, since memory limits your batch. Quantize to fit, then measure quality on your own task. Compare the true total, including people.

> *Before: generation and the KV cache. HLD M21D designs a large serving fleet, which isn't repeated here. Teaches: when self-hosting is justified, memory sizing, quantization, serving engines, utilization economics, and operations. Used by: AI10's gateway routes.*

The hook: free weights, expensive hours

The arithmetic a team usually skips. **Server price, throughput and salary are my assumptions.** Haiku 5.5's API price is from the pricing page. Real output from `QuantLab.java`:

```
Monthly comparison at 500 input and 150 output tokens a request,
Haiku 5.5 at $0.10 and $0.50, against 2 servers at $3/hour plus 1 engineer:
  API: $0.000125 a request. Self-host fixed: $19,380 a month
      1,000,000 requests: API $      125, self-host at least $19,380
     10,000,000 requests: API $    1,250, self-host at least $19,380
    100,000,000 requests: API $   12,500, self-host at least $19,380
  1,000,000,000 requests: API $  125,000, self-host at least $19,380
  break-even: about 155,040,000 requests a month
```

Below about 155 million requests a month, in this example, the API costs less. And two servers may not carry that load, so the real break-even is higher. It also compares a small hosted model with whichever open model you've chosen, and their quality may differ. Larger API models move the break-even down, as they cost more per token.

Intuition: buying or renting a van

A van rental charges by the day, and someone else services it. Buying one is cheaper if you drive it constantly. If it sits in the yard most of the week, you're paying for nothing, and also paying a mechanic.

GPUs work the same way. Some firms should own vans, because they need a custom fit-out or go where rentals can't. That's an honest reason, and "vans are free once you own them" isn't.

Mechanism

Reasons, in order of honesty

Reason	Strength	Test it
Data can't leave your environment, by rule	Strong	Are cloud platforms in your own account enough? (AI10 L6)
Air-gapped or offline	Strong	
Sustained, very high volume on a narrow task	Moderate	Do the arithmetic, including people
Fine-tuned weights you must own	Moderate	Do you need them? (L1)
Latency under your control, on your network	Moderate	Measure the API first
Independence from a vendor	Moderate	Cost of the second platform (AI10 L6)
"It's free"	**Weak**	See the hook

Embeddings are a different case. Embedding models are small and run well on modest hardware (AI04 L1), so self-hosting those is a much smaller decision.

Licences. "Open" models come with varied licences, some with usage limits. Read the licence for each model. I haven't reviewed specific ones for this course, and that's a legal question.

Memory: the first constraint

HLD M21D explains the KV cache. Here's the sizing, in two parts.

Weights are parameters times bytes. This matches the Hugging Face docs' example of an 8B model at about 16 GB in bf16, about 8 GB at 8 bits, and about 4–5 GB at 4 bits.

```
Weight memory (parameters x bytes, weights only):
    8B parameters: 16-bit    16 GB, 8-bit     8 GB, 4-bit     4 GB
   70B parameters: 16-bit   140 GB, 8-bit    70 GB, 4-bit    35 GB
  405B parameters: 16-bit   810 GB, 8-bit   405 GB, 4-bit   203 GB
```

KV cache grows with sequence length, and with concurrent requests. For an invented shape of 32 layers, 16 heads and a head dimension of 128:

```
KV cache for one sequence, 2 x layers x heads x head dim x bytes:
    2,000 tokens at 16-bit:   0.5 GB. 20 concurrent:   10.5 GB
   32,000 tokens at 16-bit:   8.4 GB. 20 concurrent:  167.8 GB
  128,000 tokens at 16-bit:  33.6 GB. 20 concurrent:  671.1 GB
```

Long contexts multiply by concurrency, so memory sets your batch size, and your batch size sets your cost per token. A model that "fits" with one short prompt may serve very few long requests.

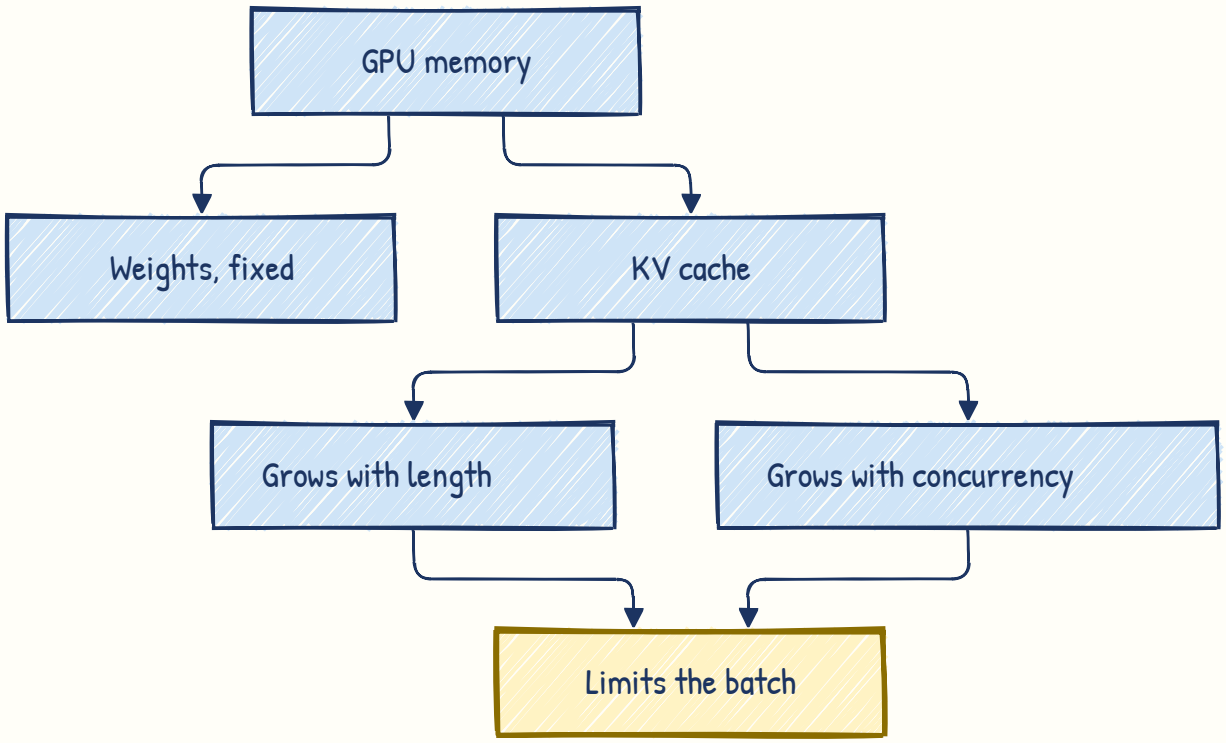


Figure 1. Weights are fixed. The KV cache grows with length and concurrency, and what's left decides how many requests can run.

Quantization

Quantization stores weights in fewer bits. Each method trades memory, speed and accuracy differently, as the Hugging Face docs put it.

The simplest kind maps each weight to the nearest of a few evenly spaced levels. Here's that on a small network I trained, with real rounding. **It's a toy with about 300 weights, so it isn't a prediction for an LLM.**

```
Rounding the trained weights of a small network (a toy):
  float      accuracy  80.6%
  16-bit     accuracy  80.6%
   8-bit     accuracy  80.5%
   6-bit     accuracy  80.4%
   4-bit     accuracy  80.9%
   3-bit     accuracy  81.0%
   2-bit     accuracy  50.0%
```

Differences above 2 bits are noise, as a higher score at 3 bits shows. At 2 bits, the method collapses: with so few levels, most weights round to zero. It's a cliff rather than a slope, in this toy, and only testing shows where yours is.

Choice	What to know, from the docs
Bit width	Fewer bits cost some accuracy
Method	Transformers integrates more than 20 methods. Some only load pre-quantized checkpoints
Calibration	Most don't need a dataset, and GPTQ does
GGUF	One file with weights, config and tokenizer. **Most GGUF models are dequantized to full precision when loaded in Transformers**, so they don't save memory there

That last row is a real trap: a small file doesn't guarantee small memory, depending on the runtime. The KV cache can also be quantized, the docs note.

Measure on your task, using the evals from AI10 L3. Don't rely on someone's benchmark. Rare segments and long inputs may degrade first.

Serving engines

You won't write this layer. An inference server handles batching, the cache and streaming, and exposes an HTTP API.

The vLLM paper (Kwon et al.) is a good example of why engines matter. Its observation is that a poorly managed KV cache wastes memory through fragmentation and duplication, which limits batch size. Its PagedAttention, inspired by virtual memory, aims for "near-zero waste in KV cache memory." The authors report 2-4× throughput at the same latency, against the systems they compared with. **These are the authors' figures from their paper**, and I ran no serving engine for this course, so measure on your own hardware.

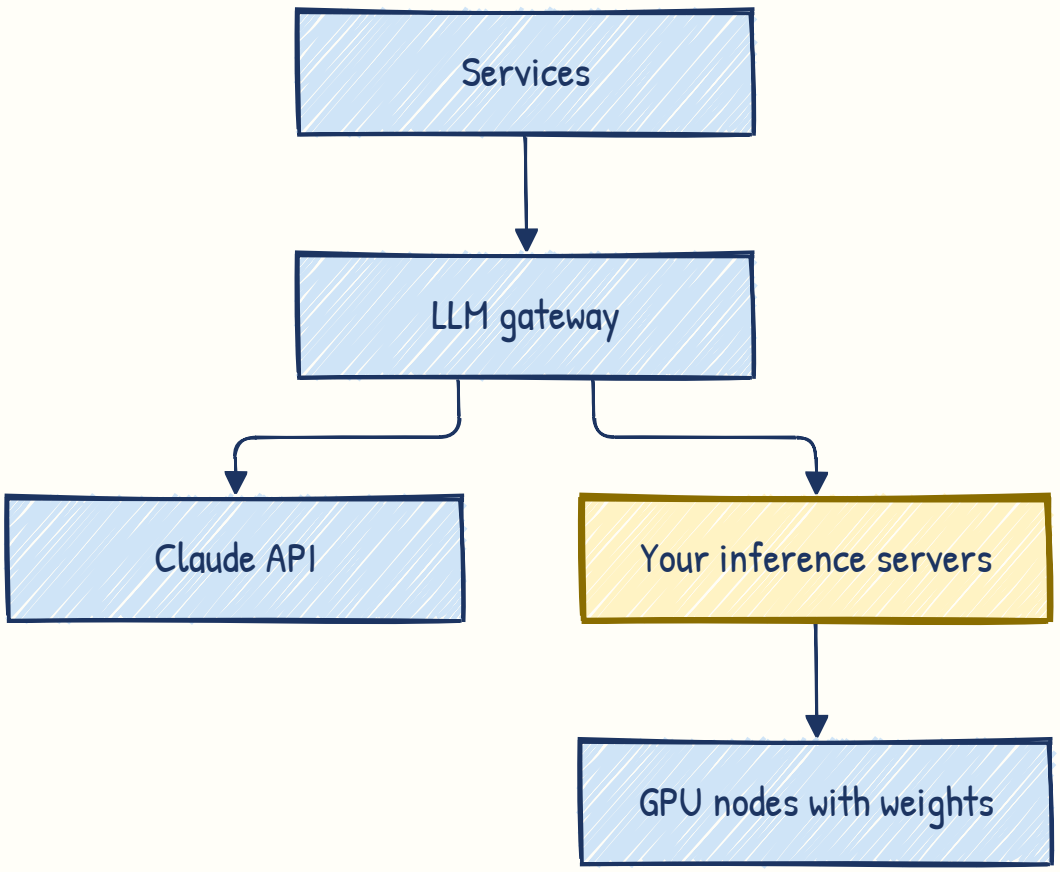


Figure 2. Your services don't change. The gateway routes a model name to an API, or to your own inference servers.

Utilization decides the cost

A server costs the same busy or idle. Using my assumptions of $3 an hour and 2,000 tokens a second:

```
Self-hosting, per million output tokens. ASSUMED: $3/hour per GPU server,
and 2,000 tokens a second when busy, ignoring engineers and input tokens:
   5% busy: $8.33
  20% busy: $2.08
  50% busy: $0.83
  90% busy: $0.46
```

An 18-fold range, from the same server. Interactive traffic has peaks and quiet nights, so you'll hold headroom. Sizing for peak means low average use, unless you can move batch work into quiet hours. An API has done that averaging for you, across its customers.

Operating it

Here's what you're signing up for:

Task	Note
Capacity	GPUs can be scarce, and slow to obtain
Cold starts	Loading large weights takes a while, so scaling up isn't instant
Upgrades	New models and engine versions, evaluated each time
Safety	Filters and abuse handling are yours now (AI09 L5)
Model files	Supply chain risk. Prefer safe formats, and verify sources. General knowledge, not read here
Monitoring	Queue depth, KV cache use, GPU memory, and first-token time
On-call	At night, it's you

Feature gaps. Prompt caching, batch pricing, tool use quality, citations and server tools may need rebuilding, or may not exist. List the features you rely on before moving.

Why the obvious fixes fall short

Obvious idea	Why it falls short
The weights are free	GPUs and people aren't
Size for weights only	The KV cache may be larger
Quantize as far as possible	There's a cliff, and quality varies by segment
Trust a benchmark	Your task and traffic differ
A GGUF file is small, so memory is small	Often dequantized on load
Scale up on demand	Cold starts are slow, and GPUs scarce
Self-host for privacy, without checking options	A cloud platform in your account may be enough

How Anthropic and the field approach it

Anthropic doesn't release Claude's weights in anything I've read, and so self-hosting Claude isn't an option. What it offers for control is deployment on cloud platforms, data residency and retention arrangements (AI09 L4, AI10 L6). **If the real need is data control, compare those before buying GPUs.**

The field. vLLM's PagedAttention, Hugging Face's quantization methods, and QLoRA are published methods. HLD M21D covers large-fleet batching and routing.

The production architecture

This is a recommended reference design, not a specific company's system.

Baseline: one server, running one model.

Improved: a serving engine behind a load balancer. New problem: it's sized for peak and idle most of the day, and every model upgrade is a project.

Production: route by need, and fill the quiet hours.

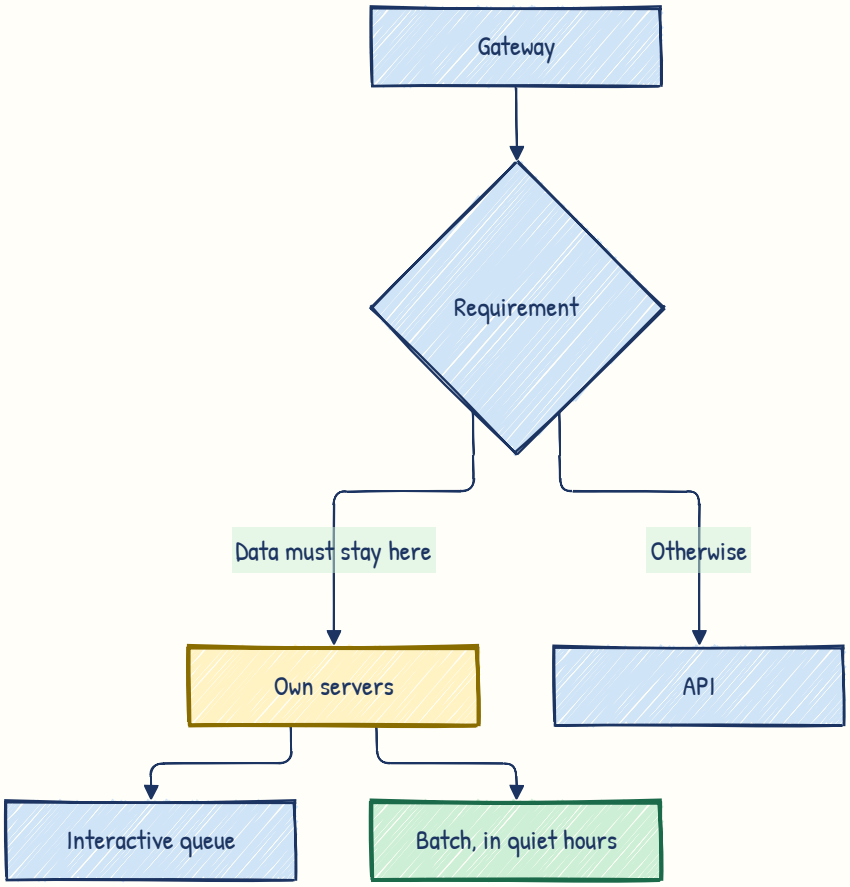


Figure 3. Interactive work gets priority. Batch work fills the idle hours, which is where the utilization comes from.

Component	Why it exists	How it communicates
Gateway	One interface for callers	HTTP (AI10 L7)
Inference servers	Batching, KV cache and streaming	HTTP or gRPC
Model store	Verified weights and adapters	Object storage
Autoscaler	Follows demand, with warm spares	Queue depth
Priority queues	Interactive before batch	Scheduler
Eval pipeline	Each model and quantization is a release	CI (AI10 L3)
GPU monitoring	Memory and queue signals	Metrics

Failures:

Failure	How you'd notice	What to do
Out of GPU memory	Crashes on long prompts	Cap context per request, and limit concurrency
Queue builds	First-token time climbs	Shed, or route to API overflow
GPU failure	Node loss	Spares, and reschedule
Slow cold start	Autoscale lags traffic	Warm pool
Quality drop after quantizing	Evals by segment	Use more bits
Compromised model file	Provenance checks	Safe formats, and verify hashes

Java

The quantizer is a few lines, and it makes the cliff visible:

```
double levels = Math.pow(2, bits - 1) - 1;
double step = max / levels;
```

At 2 bits, `levels` is 1, so a weight rounds to zero, or to the largest value. That's what collapses the network.

The cost formula shows why idle time matters:

```
return dollarsPerHour / (tokensPerSecond * busy * 3600) * 1e6;
```

In Java, a self-hosted model is an HTTP service. AI02 L3 said so. Run a dedicated server, with your Spring service as a client. Spring AI or the JDK's client can reach it (AI07 covered frameworks). I haven't run one here.

Trade-offs

Decision	Benefit	Cost
Self-host	Control, and possibly lower cost at scale	GPUs, people, and missing features
API	No operations, newest models	Per-token cost, and vendor limits
Quantize more	Fits on less hardware	Quality risk
Larger batch	Lower cost per token	Higher latency
Warm spares	Handles spikes	Idle cost
Cloud platform in your account	Data control, no GPUs	Their features and limits

When not to: unless you have a hard data constraint, or demonstrated sustained volume. Do the arithmetic in the hook with your own numbers first, and include the on-call rota.

Common mistakes

Comparing GPU hours with token prices at 100% utilization. Real traffic has quiet hours, and 5% busy costs 18 times what 90% does.

Common mistakes

Quantizing, then testing with the demo prompt. Rare segments and long inputs can degrade first. Use the eval suite.

What to say in an interview

"I'd self-host for control, such as a hard data rule or offline use, and I'd challenge cost as the reason. I compute total cost, including people and utilization, since idle GPUs still bill. I size memory for weights and KV cache, because long contexts times concurrency set the batch. I quantize to fit, then measure quality by segment. And I'd keep an API route for overflow."

Certification lens

How this lesson maps to CCAR-F

Out of scope. The exam covers Claude's API and tools. Self-hosting, quantization and serving engines aren't in the guide.

These are original practice questions, not real or leaked.

Q Q1 · A 70B model at 8 bits fits on your 80 GB GPU. Why might it still serve poorly?

A Weights take 70 GB, which leaves about 10 GB for the KV cache and runtime overhead. By my example shape, that's only one long request or about 20 short ones. Memory limits the batch, so throughput falls and cost per token rises. It also needs an eval to confirm quality at 8 bits.

Q Q2 · Your servers cost $6 an hour each, and produce 3,000 tokens a second when busy. What does a million output tokens cost at 25% and at 75% busy?

A At 25%: $6 ÷ (3,000 × 0.25 × 3,600) × 1,000,000 = about $2.22. At 75%: about $0.74. It's triple the cost for the same server, and it leaves out input tokens and engineers.

Q Q3 · Legal says customer data "can't go to a third party." List what you'd check before deciding to self-host.

A Whether that rule excludes a cloud provider in your own account, what the retention and ZDR options cover, whether data residency is the real need, and who the data processor is per platform (AI09 L4, AI10 L6). Then compare with the cost of self-hosting, including people, plus features you'd lose. Get the requirement from legal in writing.

Revision - Lesson 2

→ Self-host for control. Cost alone is usually a weak reason.

→ Compare true totals, including people. Hook's break-even: about 155M requests a month, under its assumptions.

→ Weights: parameters times bytes. KV cache grows with length and concurrency.

→ Memory sets batch size, and batch size sets cost per token.

→ Quantization saves memory, with a quality risk. Measure by segment. There can be a cliff.

→ Most GGUF models are dequantized on load in Transformers.

→ vLLM's PagedAttention reports 2-4× throughput, from reducing KV cache waste, per its paper.

→ 5% busy costs 18 times what 90% busy does.

→ Check cloud platforms, residency and retention options first if the need is data control.

→ You lose provider features, and you gain capacity, upgrades, safety and on-call.

Your notes